Process Creation & Threads
(PCC-CSE 491: Operating System Lab)
Day 3 Lab Work

Course Coordinators
Department of Computer Science and Engineering
RCC Institute of Information Technology, Kolkata (West Bengal)

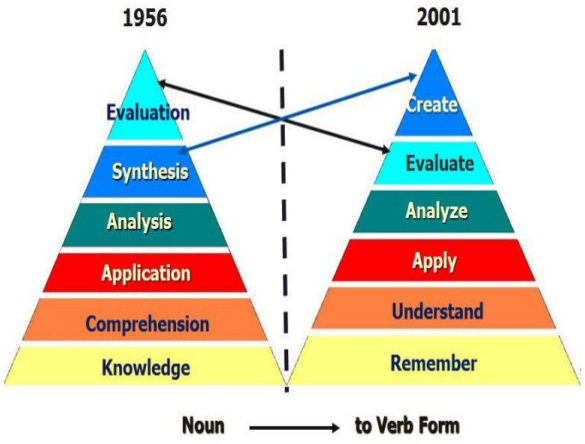
Agenda

Exp-3 **(Day-3)**	1. Demonstrate the creation a process using fork(() 2. Demonstrate the creation multiple processes using fork(() 3. Demonstrate the parent & child process using fork(() 4. Demonstrate use of pthread(POSIX)	**CO2**	**PCA 1**

Course Outcomes

CO#	Outcome Statements	Knowledge Level
CO1	**Understanding** of different Unix/Linux commands and shell programming	**Understanding (Level II)**
CO2	**Demonstrate** the creation of processes and POSIX threads.	**Applying (Level III)**
CO3	**Illustrate** the implementation of process scheduling and process synchronization (Signal and Semaphore)	
CO4	**Analyse** the deadlock avoidance and detection algorithms.	**Analysing (Level IV)**
CO5	**Determine** different Memory allocation and File accessing techniques.	**Evaluating (Level V)**
CO6	**Develop** Inter-Process Communication through system calls.	**Creating (Level VI)**

Bloom's taxonomy is **a classification system used to define and distinguish different levels of human cognition**—i.e., thinking, learning, and understanding.



Bloom's taxonomy was developed to **provide a common language for teachers to discuss and exchange learning and assessment methods**.

Introduction to Process

- The term "**process**" was first used by the designers of the **MULTICS** (Multiplexed Information and Computing Service) in 1960's.
- A process is a program at the time of execution; process execution must progress in sequential fashion
- A process is the more than the program code, it includes the

(i) program counter,

(ii) the process stack,

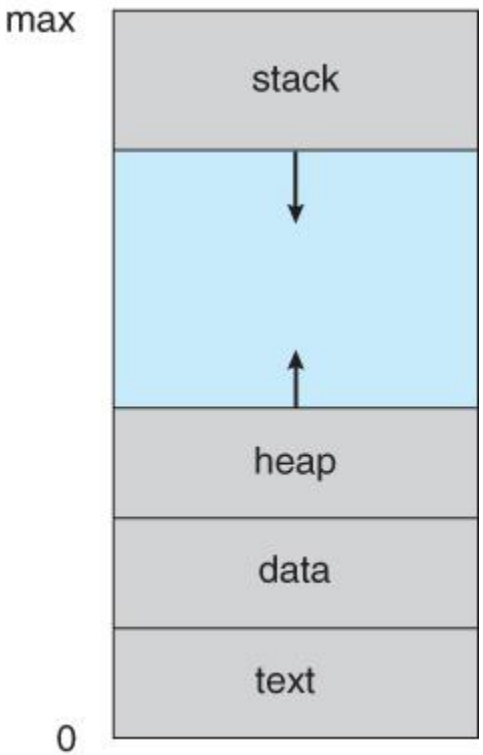
(iii) the contents of process register and

(iv) value of the variable.

Represented Process

Process memory is divided into four sections for efficient working :

- The **Text section** is made up of the compiled program code, read in from non-volatile storage when the program is launched.
- The **Data section** is made up of the global and static variables, allocated and initialized prior to executing the main.
- The **Heap** is used for the dynamic memory allocation and is managed via calls to new, delete, malloc, free, etc.
- The **Stack** is used for local variables. Space on the stack is reserved for local variables when they are declared.



A process is represented by the tuple

Process < Pid, Pcode, Pdata, Prv, Pcv>

Where

Pid is a process id is unique in the system., **Pcode** is a program code.

Pdata is a data used during execution., **Prv** is a register values are the value in the machine register., **Pcv** is a program counter value.

Process Operations

A process is created by the OS kernel in response to a user command. When a new process is created then the OS performs the following activities –

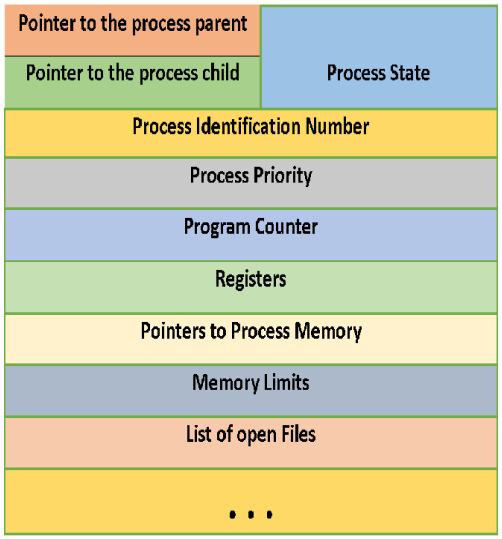
i) Creates a PCB for the process

ii) Assigns process id and priority

iii) Allocates memory and other resources to the process

iv) Sets up the process environment

v) Initializes resource according information for the process.



PCB

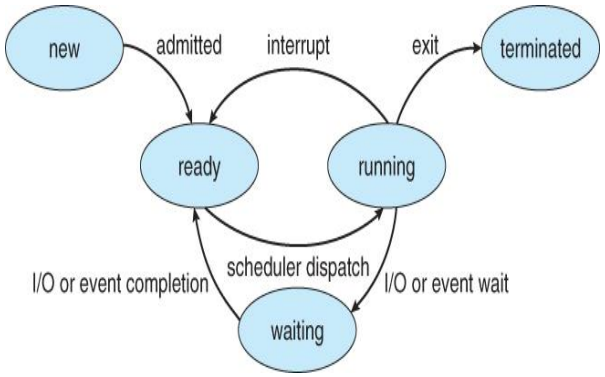
Operations on Process

i) Process creation

ii) Process states

iii) Process state transitions

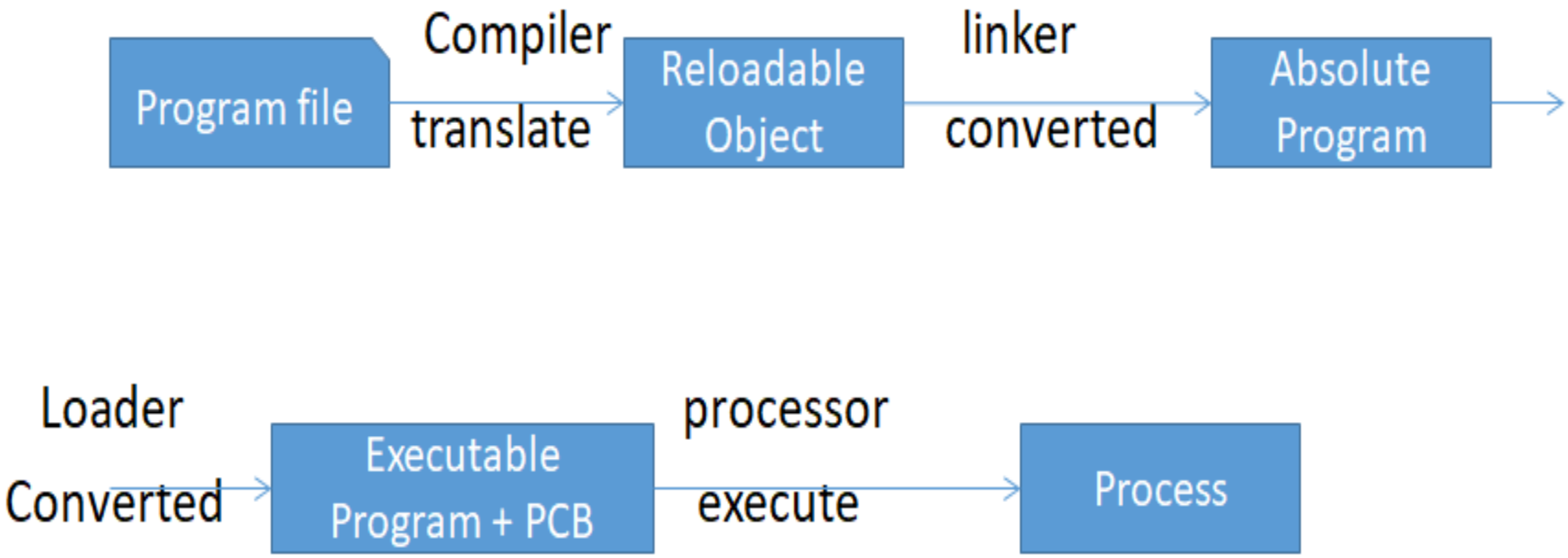
iv) Process termination



Process States

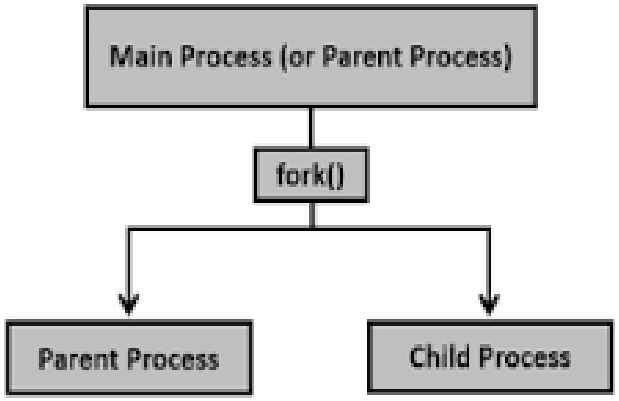
Process Creation

When a user initiates to execute a program then the OS creates a process to represent the execution of this program.



Creation of a process using fork () systems

1. Demonstrate the creation a process using fork(()




```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
      // make two process which run same
      // program after this instruction
      fork();

      printf("Hello world!\n");
      return 0;
}
```

Predict the Output of the above program ……!!!.

Multiple Process Creation: fork()

2. Demonstrate the creation multiple processes using fork(()

```
#include <stdio.h>
#include <sys/types.h>
int main()
{
 fork();//Line 1          L1    // There will be 1 child process
                        /    \  // created by line 1.
 fork();//Line 2       L2      L2 // There will be 2 child processes
 fork();//Line 3     /  \    /  \ //  created by line 2
                    L3  L3  L3  L3  // There will be 4 child processes
 printf("hello\n");                 // created by line 3
 return 0;
}
```

Predict the Output of the above program ……!!!.

Multiple Process Creation: fork()

```
#include <stdio.h>
#include <sys/types.h>
int main()
{
      fork();
      fork();
      fork();
      printf("hello\n");
      return 0;
}
```

Output:….?????

Explaination

hello hello hello hello hello hello hello hello

// The number of times 'hello' is printed is equal to number of process created.

//Total Number of Processes = 2^n, where n is number of fork system calls. So here n = 3, $2^3 = 8$

The main process: P0

Processes created by the 1st fork: P1

Processes created by the 2nd fork: P2, P3

Processes created by the 3rd fork: P4, P5, P6, P7

So there are total eight processes (new child processes and one original process).

Parent & Child Process Creation

3. Demonstrate the parent & child process using fork(()

```
#include <sys/types.h>
#include <unistd.h>
void forkexample()
{
 int x = 1, p_id, p_pid;
        // child process because return value zero
        if (fork() == 0){
            p_id=getpid();
            printf("Child has pid =%d, x=%d\n", p_id, ++x);
            }
        // parent process because return value non-zero.
        else {
            p_pid=getpid();
            printf("Parent has pid =%d, x=%d\n", p_pid, --x);
} }
int main()
{
        forkexample();
        return 0;
}
```

Predict the Output of the above program ……!!!.

Parent & Child process creation

Output:(Explanation)

Parent has pid = 123, x = 0
Child has pid = 324 , x = 2
(or)
Child has pid=324, x = 2
Parent has pid 123, x = 0

```
#include <sys/types.h>
#include <unistd.h>
void forkexample()
{
 int x = 1, p_id, p_pid;
        // child process because return value zero
  if (fork() == 0){
   p_id=getpid();
   printf("Child has pid =%d, x=%d\n", p_id, ++x);
  }
   // parent process because return value non-zero.
  else {
   p_pid=getpid();
  printf("Parent has pid =%d, x=%d\n", p_pid, --x);
} }
int main()
{
        forkexample();
        return 0;
}
```

Here, two outputs are possible because the parent process and child process are running concurrently.

So we don't know whether the OS will first give control to the parent process or the child process.

In the code, a child process is created. fork() returns 0 in the child process and positive integer in the parent process.

Here, global variable change in one process does not affected two other processes because data/state of two processes are different. And also parent and child run simultaneously so two outputs are possible.

Threads

What is a Thread?

A thread is a path of execution within a process. A process can contain multiple threads.

A process is divided into numbers of light weight process, each light weight process is said to be a thread.

- The thread has a program counter that keeps the tracks of which instruction to execute next, it has register, which hold its current working variables.
- A thread is a basic unit of program execution.
- A thread can be defined as a process in execution within program.
- A thread is a single sequential flow of control within a program.
- The idea is to achieve parallelism by dividing a process into multiple threads.

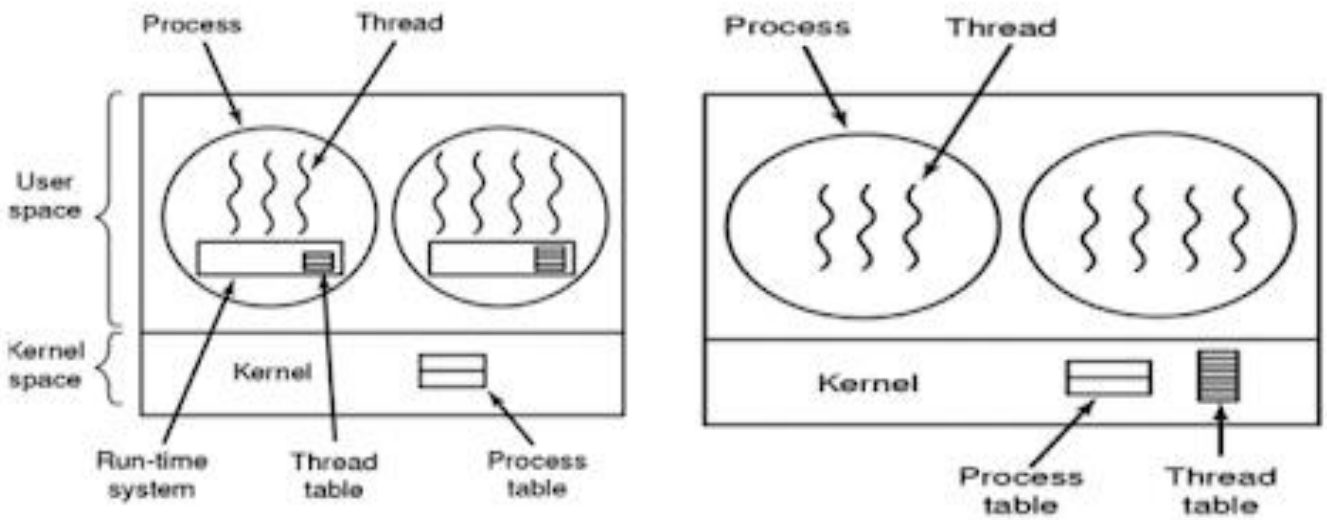
For example, in a browser, multiple tabs can be different threads. MS Word uses multiple threads: one thread to format the text, another thread to process inputs, etc.

Thread Implementation

There are two main ways to implement threads-

a) User level threads (in user space)

b) Kernel level threads (in the kernel space)



(a) A user-level threads package. (b) A threads package managed by the kernel.

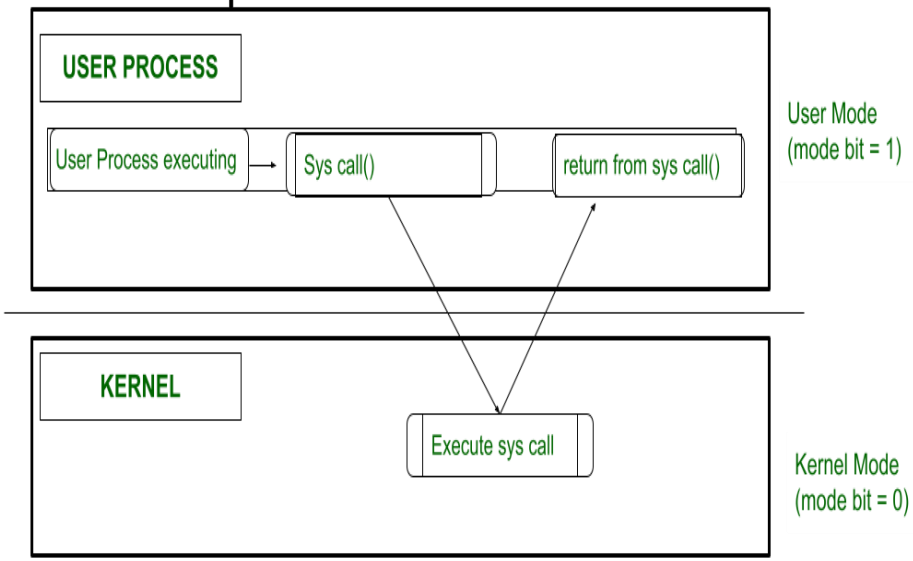
Thread Implement in User Space

A thread is an execution unit, which is part of a process. It shares the process's resources. It is scheduled by the scheduler.

There are two ways to implement a thread, they're either in user space or in the Kernel.

Threads can be implemented in user space, without the support of the kernel. The following things happen if we implement a thread in user space –

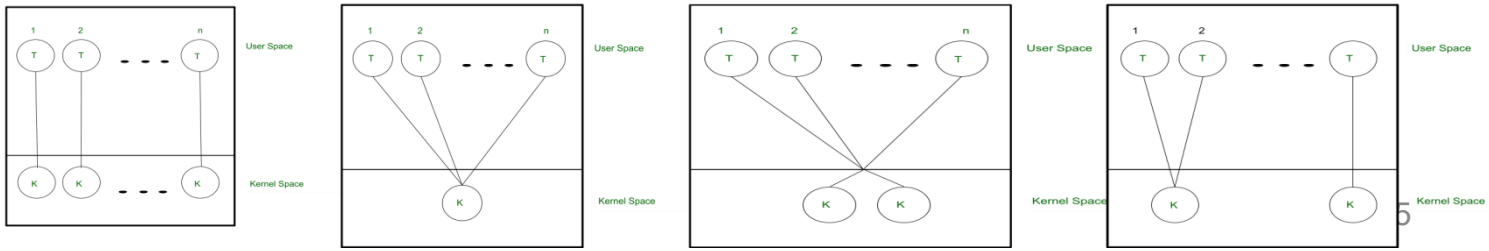
- The corresponding code and the data structures used are stored in the user space.
- If an API is invoked, it results in a local system call in user space, rather than a system call.
- The threads are managed entirely by the run-time system, without the kernel knowing anything about it.



User-level thread models :

A thread in the userspace can be implemented using any of the following four methods:-

- One-to-one
- Many-to-one
- Many-to-many
- Two-level.



POSIX Threads in OS

- The POSIX thread libraries are a C/C++ thread API based on standards.
- It enables the creation of a new concurrent process flow.
- It works well on multi-processor or multi-core systems, where the process flow may be scheduled to execute on another processor, increasing speed through parallel or distributed processing.
- To utilise the PThread interfaces, we must include the header pthread.h at the start of the C program - **`#include <pthread.h>`**
- PThreads is a highly concrete multithreading system that is the UNIX system's default standard.
- PThreads is an abbreviation for POSIX threads, and POSIX is an abbreviation for Portable Operating System Interface, which is a type of interface that the OS must implement.
- PThreads in POSIX outline the threading APIs that the OS must provide.

Why is Pthreads used?

- The fundamental purpose for adopting Pthreads is to improve programme performance.
- When compared to the expense of starting and administering a process, a thread requires far less OS overhead. Thread management takes fewer system resources than process management.
- A process's threads all share the same address space. Inter-thread communication is more efficient and, in many circumstances, more user-friendly than IPC.
- Threaded applications provide possible performance increases and practical advantages over non-threaded programmes in a variety of ways.
- Multi-threaded programmes will run on a single-processor system but will automatically make use of a multiprocessor machine without the need for recompilation.
- The most significant reason for employing Pthreads in a multiprocessor system is to take advantage of possible parallelism.
- In order for a programme to use Pthreads, it must be divided into discrete, independent tasks that may run concurrently.

Demonstrate use of pthread(POSIX)

multithread.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
void *myThreadFun(void *vargp)
{
   sleep(1);
   printf("Printing OS Lab from Thread \n");
   return NULL;
}

int main()
{
pthread_t thread_id;
printf("Before Thread\n");
pthread_create(&thread_id, NULL, myThreadFun, NULL);
pthread_join(thread_id, NULL);
printf("After Thread\n");
exit(0);
}
```

POSIX Threads (or Pthreads) is a POSIX standard for threads. Implementation of pthread is available with gcc compiler.

//Unistd.h - provides access to the POSIX operating system API.
//pthread.h file contains function declarations and mappings for threading interfaces and defines a number of constants used by those functions.

//**Sleep()** in order to wait for a current thread for a specified time. (1 second)

//thread_id, which is of type pthread_t, which is an integer used to identify the thread in the system. pthread_t is the data type used to uniquely identify a thread.

//create a thread & takes 4 arguments

1st argu. - pointer to **thread_id**
2nd argu - specifies attributes (default NULL)
3rd argu – thread to be created - **myThreadFun**
4th argu - used to pass argus to myThreadFun - **NULL**

//**pthread_join()** for threads is the equivalent of wait() for processes. A call to pthread_join blocks the calling thread until the thread with identifier equal to the first argument terminates.

How to compile pthread program?

To compile a multithreaded program using gcc

```
$ gcc multithread.c -lpthread
$ ./a.out
Before Thread
Printing OS Lab from Thread
After Thread
$
```

References

1. https://youtu.be/PwxTbksJ2fo
2. https://youtu.be/pSW9d3Oaie8
3. https://youtu.be/9cuwgiHQyoI
4. https://youtu.be/IFEFVXvjiHY
5. OS Lab manual

Thank you!!!!